

Assignment 11

Q1

Show that the following languages are in NP:

- 3SAT = $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 3CNF formula} \}$. A 3CNF formula is an AND of clauses where each clause is an OR of 3 literals (variables).
- VERTEX COVER = $\{ \langle G, k \rangle \mid G \text{ is an undirected graph that has a } k - \text{node vertex cover} \}$. (A *vertex cover* is a subset of nodes such that every edge of the graph G touches one of the nodes).
- TSP = $\{ \langle G, W, k \rangle \mid G \text{ is an undirected graph, with weights } w_{i,j} \text{ on each edge that has a tour of length at most } k \text{ that visits every node exactly once} \}$
- MAX-CUT = $\{ \langle G, k \rangle \mid G \text{ has a cut of size } k \text{ or more} \}$. A *cut* in an undirected graph is a separation of the vertices V into two disjoint subsets S and T . The size of the cut is the number of edges that have one endpoint in S and one in T .
- 3COLOR = $\{ \langle G \rangle \mid G \text{ is colourable with 3 colors} \}$. A *colouring* of a graph is an assignment of colours to the nodes such that no node is adjacent to another node with the same colour.

A1

- a) 3SAT = $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 3CNF formula} \}$. A 3CNF formula is an AND of clauses where each clause is an OR of 3 literals (variables).

NTM:

"On input $\langle \phi \rangle$ where ϕ is a 3CNF formula,

1. Nondeterministically assign True/False to each variable.
2. Test whether ϕ is True under the currently assigned value.
3. Accept if yes, otherwise reject."

Verifier in polynomial time:

"On input $\langle \phi, v \rangle$:

1. Test whether v is a list that gives T/F value assignment of each variable in ϕ . If not, reject.
2. Change the variables of ϕ into T/F value according to v .
3. Test whether ϕ is TRUE. Accept if yes. Reject if not."

If we consider the RAM model, then Step 1 and Step 2 runs in time $O(n)$. Step 3 is also $O(n)$ time, since for each clause, it needs $O(1)$ time to get T/F value, and ϕ contains $O(n)$ clauses. In the end, $O(n)$ ANDs connecting all clauses needs $O(n)$ time.

Overall, the verifier runs in $O(n)$ time when using the RAM model. Then also a 1-tape TM can do the same in polynomial time.

VERTEX COVER = $\{ \langle G, k \rangle \mid G \text{ is an undirected graph that has a } k\text{-node vertex cover} \}$. (A *vertex cover* is a subset of nodes such that every edge of the graph G touches one of the nodes).

NTM:

"On input $\langle G, k \rangle$ where G is an undirected graph,

1. Nondeterministically select a subset c of k nodes in G .
2. Test whether each edge in G starts or ends with the node in c .
3. Accept if yes, otherwise reject."

Verifier in polynomial time:

On input $\langle \langle G, k \rangle, c \rangle$:

1. Test whether c is a subset with k nodes in G .
2. Test whether all edges in G have at least one vertex in c .
3. Accept if both yes, otherwise reject.

c) TSP = $\{ \langle G, W, k \rangle \mid G \text{ is an undirected graph, with weights } w_{i,j} \text{ on each edge that has a tour of length at most } k \text{ that visits every node exactly once} \}$

NTM:

"On input $\langle G, W, k \rangle$:

1. Nondeterministically select a path p in G .
2. Check whether p visits every node in G exactly once.
3. Calculate the total weights of p and check whether it is no more than k .
4. Accept if both yes, otherwise reject."

Verifier in polynomial time:

"On input $\langle \langle G, w, k \rangle, c \rangle$ (say G has m nodes),

1. Test whether c is a sequence of m distinct nodes in G .
2. Test whether there is an edge between (c_i, c_{i+1}) , $\forall 0 \leq i < m - 1$.
3. Calculate $cost = \sum_{i=0}^{m-2} w_{c_i, c_{i+1}}$, accept if $cost \leq k$, otherwise reject."

d) MAX-CUT = $\{ \langle G, k \rangle \mid G \text{ has a cut of size } k \text{ or more} \}$. A *cut* in an undirected graph is a separation of the vertices V into two disjoint subsets S and T . The size of the cut is the number of edges that have one endpoint in S and one in T .

NTM:

"On input $\langle G, k \rangle$ where G is an undirected graph:

1. Nondeterministically divide the nodes of G into two disjoint subsets S and T .

2. Calculate the number of edges in G that have one endpoint in S and one in T and check whether the number is k or more.
3. Accept if yes, otherwise Reject."

Verifier in polynomial time:

"On input $\langle G, k, c \rangle$ where G is the graph,

1. Test whether c is the subset of nodes in G .
2. For each edge in G , if one endpoint is in c and the other is not, add 1 to the counter.
3. Accept if the counter is equal or greater than k , otherwise reject."

$3\text{COLOR} = \{ \langle G \rangle \mid G \text{ is colourable with 3 colors} \}$. A *colouring* of a graph is an assignment of colours to the nodes such that no node is adjacent to another node with the same colour.

NTM:

"On input $\langle G \rangle$ where G is the graph,

1. Nondeterministically assign each node with a color 1/2/3.
2. For each edge in G , test whether two endpoints have different colors.
3. Accept if yes, otherwise reject."

Verifier in polynomial time:

"On input $\langle G, c \rangle$:

1. Test whether c is an assignment of colors to all the nodes of G .
2. Test whether there is no node adjacent to another node with the same color.
($O(n^2)$ time)
3. Accept if both yes, otherwise Reject."

Q2

Show that NP is closed under union, concatenation and Star.

A2

Assume $A \in NP$ and $B \in NP$. Then there exist polynomial-time NTMs T_A and T_B deciding A and B , respectively.

Let $t_A(n)$ and $t_B(n)$ be polynomial upper bounds on the running times of T_A and T_B on inputs of length at most n .

Union

We construct an NTM T_U deciding $A \cup B$.

$T_U =$ "On input s :

1. Nondeterministically choose one of the following two branches:

- Simulate T_A on s .

- Simulate T_B on s .

2. Accept if the chosen simulation accepts; otherwise reject."

If $s \in A \cup B$, then $s \in A$ or $s \in B$, so there exists an accepting branch of $T_\cup$.

If $s \notin A \cup B$, then $s \notin A$ and $s \notin B$, so all branches reject.

The running time is bounded by

$$O(\max\{t_A(n), t_B(n)\}),$$

which is polynomial. Therefore,

$$A \cup B \in NP.$$

Concatenation

We construct an NTM $T_\circ$ deciding $A \circ B$.

$T_\circ$ = "On input s :

- 1. Nondeterministically choose a split $s = xy$, where x and y may be empty.

- 2. Simulate T_A on x .

- 3. Simulate T_B on y .

- 4. Accept if both simulations accept; otherwise reject."

If $s \in A \circ B$, then there exist strings $x \in A$ and $y \in B$ such that $s = xy$. Then $T_\circ$ has an accepting branch choosing this split.

If $s \notin A \circ B$, then for every split $s = xy$, either $x \notin A$ or $y \notin B$, so every branch rejects.

The running time is bounded by

$$O(t_A(n) + t_B(n) + n),$$

which is polynomial. Therefore,

$$A \circ B \in NP.$$

Star

We construct an NTM T_* deciding A^* .

T_* = "On input s :

- 1. If $s = \epsilon$, accept.

- 2. Otherwise, nondeterministically choose an integer r with $1 \leq r \leq |s|$.

- 3. Nondeterministically split s into r nonempty substrings

$$s = s_1 s_2 \cdots s_r.$$

- 4. For each $i = 1, \dots, r$, simulate T_A on s_i .

- 5. Accept if all simulations accept; otherwise reject."

If $s \in A^*$, then s can be written as

$$s = s_1 s_2 \cdots s_r$$

where each $s_i \in A$. If $s \neq \epsilon$, we may choose the decomposition so that all s_i are nonempty and $r \leq |s|$. Hence T_* has an accepting branch.

If $s \notin A^*$, then no such decomposition exists, so every branch rejects.

The running time is bounded by

$$O(n \cdot t_A(n) + n),$$

where $n = |s|$, which is polynomial. Therefore,

$$A^* \in NP.$$

Thus, NP is closed under union, concatenation, and star.

Q3

Say whether the following are true or not and argue why:

- $P \subseteq NP$
- P is closed under complementation i.e., if $A \in P$ then $\bar{A} \in P$
- $\overline{SAT} \in NP$
- $P \in NP \cap coNP$
- If $P = NP$ then $NP = coNP$

A3

a)

True

As the deterministic Turing machine is a specific type of nondeterministic Turing machine,

$$TIME(t(n)) \in NTIME(t(n))$$

By definition,

$$NP = \bigcup_k NTIME(n^k), \quad P = \bigcup_k TIME(n^k)$$

b)

True

Assume $A \in P$, and T_A decides A .

Denote $t(M)$ as the upper bound of the run time of NTM M . It's easy to see $t(T_A)$ is polynomial.

We build a TM T_- to decide $\bar{A}$.

T_- = "On input s :

1. Run T_A on s .

Accept if T_A rejects, reject if T_A accepts.

We can calculate that,

$$t(T_-) = t(T_A)$$

c)

The statement

$$\overline{SAT} \in NP$$

is currently unknown.

We know that

$$SAT \in NP.$$

Therefore, by definition of $coNP$,

$$\overline{SAT} \in coNP.$$

However, whether

$$\overline{SAT} \in NP$$

is equivalent to the famous open question of whether

$$NP = coNP.$$

We first prove that $\overline{SAT}$ is $coNP$ -complete:

We say a language C is $coNP$ -complete if:

1. $C \in coNP$
2. for any $L \in coNP$, $L \leq_p C$

Obviously, $\overline{SAT} \in coNP$ because $SAT \in NP$.

For any language $L \in coNP$, $\overline{L} \in NP$, and since $SAT \in NP$ -complete, we have $\overline{L} \leq_p SAT$.

Thus by the definition of $\leq_p$, there exists a polynomial-time computable function f such that,

$$x \in \overline{L} \Leftrightarrow f(x) \in SAT$$

This is equivalent to say that,

$$x \notin \overline{L} \Leftrightarrow f(x) \notin SAT$$

, which is also equivalent to,

$$x \in L \Leftrightarrow f(x) \in \overline{SAT}.$$

Thus we have $L \leq_p \overline{SAT}$ as well. Therefore $\overline{SAT}$ is $coNP$ -complete.

Indeed, if $\overline{SAT} \in NP$, then since $\overline{SAT}$ is $coNP$ -complete, every language in $coNP$ would be in NP (because for any language $L \in coNP$, $L \leq_p \overline{SAT} \in NP$ indicates that $L \in NP$). Hence,

$$coNP \subseteq NP.$$

Then for any language $L \in NP$, we have $\overline{L} \in coNP$. Since $coNP \subseteq NP$, this implies $\overline{L} \in NP$, so $L \in coNP$. Therefore,

$$NP \subseteq coNP.$$

Thus,

$$NP = coNP.$$

Conversely, if $NP = coNP$, then since $\overline{SAT} \in coNP$, we immediately get

$$\overline{SAT} \in NP.$$

So

$$\overline{SAT} \in NP \iff NP = coNP.$$

Since it is currently unknown whether $NP = coNP$, the truth of the statement $\overline{SAT} \in NP$ is also unknown.

d)

$P \in NP \cap coNP$: True

We know that P is closed under complementation, and $P \subseteq NP$. Thus $\overline{P} \subseteq NP$ and $P \subseteq coNP$. Thus we have $P \in NP \cap coNP$.

e)

If $P = NP$ then $NP = coNP$: True

If $P = NP$:

For any $A \in NP$, $A \in P$. And we know that P is closed under complementation, so $\overline{A} \in P$ and $\overline{A} \in NP$ and $A \in coNP$. Thus $NP \subseteq coNP$.

For any $A \in coNP$, $\overline{A} \in NP$ and $\overline{A} \in P$. Thus, $A \in P$ since P is closed under complementation, and obviously $A \in NP$. Thus $coNP \subseteq NP$.

Therefore, $NP = coNP$.

Intuition for P and NP

NP = All languages where can verify membership quickly

P = All languages where can test membership quickly

Examples of quickly verifying membership:

- *HAMPATH*: Give the Hamiltonian path.
- *COMPOSITES*: Give the factor.

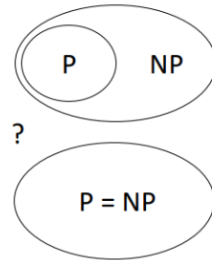
The Hamiltonian path and the factor are called **short certificates** of membership.

$P \subseteq NP$

Question: $P = NP$?

Conjecture: $P \neq NP$.

Hard to prove the conjecture because polynomial-time algorithms are powerful.



coNP

NP = All languages where can verify membership quickly

P = All languages where can test membership quickly

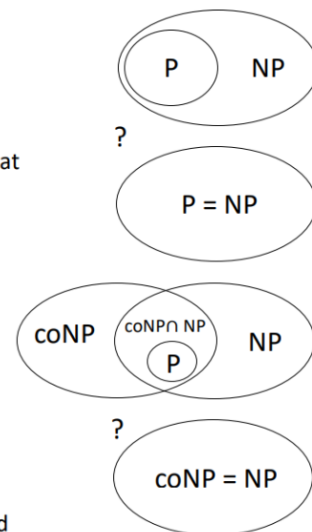
HAMPATH, CLIQUE, SUBSET – SUM are not obviously members of NP

- Verifying that something is not present seems more difficult than verifying that it's present

- **coNP** is the class which contains the languages that are complements of languages in NP

coNP = All languages that have a short "no" certificate

- Every "yes" instance of $A \in NP$ is a "no" instance of $\bar{A} \in coNP$
- We don't know whether coNP is different from NP
- Eg.,
 - PRIMES is in coNP
 - $x \notin PRIMES$: "no" certificate is simply a factorization of x .
Eg., $12 = 4 \cdot 3$
 - $P \subseteq coNP \cap NP$
 - If $L \in P$, then the algorithm to recognize $\bar{L}$ simulate the TM for L and invert the answer
 - PRIMES is also in NP !!



12

Q4

Show that $2COLOR = \{ \langle G \rangle \mid G \text{ is colourable with 2 colors} \}$ is in NP and in P.

A4

$2COLOR$ is in P , which also indicates that it is in NP :

We now give a polynomial-time decider for $2COLOR$.

$M =$ "On input $\langle G \rangle$:

1. Initially mark every vertex as uncolored.
2. For each uncolored vertex s in G :

Assign color 0 to s .

Run BFS or DFS starting from s .

Whenever we visit a vertex u , for every neighbor v of u :

- If v is uncolored, assign

$$c(v) = 1 - c(u).$$

- If v is already colored and

$$c(v) = c(u),$$

reject.

3. If the algorithm finishes without finding any conflict, accept."

We prove correctness.

First, suppose M accepts. Then during the algorithm, no edge was found whose two endpoints have the same color. Hence, for every edge $\{u, v\} \in E(G)$,

$$c(u) \neq c(v).$$

Therefore, the coloring constructed by M is a valid 2-coloring of G . Thus,

$$G \in 2COLOR.$$

Conversely, suppose $G \in 2COLOR$. Then there exists a valid 2-coloring of G . In a valid 2-coloring, every edge connects vertices of different colors. The BFS/DFS procedure assigns colors by forcing every neighbor of a vertex to have the opposite color. Therefore, if G is truly 2-colorable, this process will never force two adjacent vertices to have the same color. Hence M will not reject, and eventually it accepts.

The algorithm also handles disconnected graphs, because whenever one connected component is finished, it chooses another uncolored vertex and starts the same process again.

If we consider the RAM model, then the running time is

$$O(|V| + |E|),$$

because each vertex and each edge is processed at most a constant number of times. Hence the algorithm runs in polynomial time. Then also a 1-tape TM can do the same in polynomial time.

Therefore,

$$2COLOR \in P(NP)$$

Q5

Let $2SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 2CNF formula} \}$. Show that this language is in NP and in P.

A5

$2SAT$ is in P:

We can construct a graph by setting each variable as a node and adding edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$ if there exists a clause $(a \vee b)$.

If there exist paths $a \rightarrow \neg a$ and $\neg a \rightarrow a$, the 2CNF formula is unsatisfiable, since we cannot assign the same value to all reachable nodes, which means we cannot keep the values of all the clauses be true.

So we can construct the TM M decides $2SAT$.

$M =$ "On input ϕ :

1. If ϕ isn't in 2cnf form, reject.

2. Construct a graph $G = (V, E)$ where V is the set of nodes

$$\{x_1, x_2, \dots, x_n, \overline{x_1}, \overline{x_2}, \dots, \overline{x_n}\}.$$

For each clause $(a \vee b)$, add a direct edge from $\overline{a}$ to b and add a direct edge from $\overline{b}$ to a .

3. For each variable a :

Find if there exist paths $a \rightarrow \neg a$ and $\neg a \rightarrow a$.

Reject if both of these paths found, otherwise Accept."

This TM is in polynomial time.

More explanation:

Suppose ϕ has the form

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \cdots \wedge (a_n \vee b_n)$$

where

$$a_i, b_i \in \{x_1, x_2, \dots, x_n, \overline{x_1}, \overline{x_2}, \dots, \overline{x_n}\}.$$

If ϕ is satisfiable, then $a_i \vee b_i$ should be true for $1 \leq i \leq n$. To ensure $a_i \vee b_i$ is true:

1. If a_i is false, then b_i must be true. If a_i is true, then b_i can be true or false. We use a graph to represent it by adding a direct edge from $\overline{a_i}$ to b_i . ($\overline{a_i} \rightarrow b_i$)
2. If b_i is false, then a_i must be true. If b_i is true, then a_i can be true or false. We use a graph to represent it by adding a direct edge from $\overline{b_i}$ to a_i . ($\overline{b_i} \rightarrow a_i$)

Suppose there is an edge $a \rightarrow b$. From 1 and 2 we know that if a is true, then b must be true. If a is false, then b can be anything. On top of that, $a \rightarrow b \rightarrow c$ implies if a is true, then c must be true. If there is a path from x_i to $\overline{x_i}$, x_i must be false since x_i and $\overline{x_i}$ can't be true simultaneously. However, if both of the 2 paths $(x_i \Rightarrow \overline{x_i}, \overline{x_i} \Rightarrow x_i)$ exist, the boolean formula can never be true since x_i and $\overline{x_i}$ can't have the same value. Otherwise, there is a satisfying assignment for ϕ .